Tomasulo Report

Tomasulo Simulator with Multiple Issue - Report

Name: Dhruv Patel **ID:** 2479904

1. Introduction

The starter code was a Python Tomasulo simulator that issued one instruction per cycle. I changed it so that it can issue 1 to 4 instructions per cycle, in program order. The simulator keeps the same file layout as the starter (`main.py`, `init.py`, `issue.py`, `exe.py`, `mem.py`, `wb.py`, `commit.py`, `print_status.py`) and adds `recover.py`.

I have to be honest about one thing: the stage files are mostly rewritten, not patched. The starter had a few timing quirks (for example an extra cycle in load write-back and a rename bug when two instructions write the same register), and they made it hard to add more than one issue slot. So the width-1 cycle counts are not always identical to the starter's counts. The rules I used are listed in section 3.

2. What was added

- **Issue width 1-4** (`Issue width = N` in the input file, or the second argument of `main.py`). Up to N instructions enter the ROB, a reservation station or the load/store queue in one cycle. Issue is in order: it stops at the first instruction that finds no free ROB entry, reservation station or load/store queue slot. A branch predicted taken ends the group.
- **Register renaming** with a table that points each register to the newest ROB entry that writes it. Older writers cannot overwrite it (this fixes the write-after-write problem). Instructions in the same group can depend on each other.
- **Branch prediction:** 1-bit predictor with an 8-entry BTB (index = pc mod 8). If a branch was wrong, it is found when the branch finishes EX in cycle n, and the right instructions are fetched in cycle n+2. Wrong-path instructions are removed from the ROB, reservation stations, load/store queue and units, and the rename table is rebuilt.
- **Load/store queue:** a load takes its value from the nearest older store with the same address (1 cycle in MEM), waits if that store's data is not ready, and reads memory otherwise. With `Load speculation = on` (default) a load does not wait for older stores whose address is not known. If it was wrong, the load and everything after it is squashed and fetched again. With `off` the load waits. Stores write memory at commit.
- **In-order commit**, up to `Commit width` per cycle (default = issue width).
- **CDB width:** the number of results broadcast per cycle (default 1, as in the assignment text). Extra option so the effect of the bus can be shown.
- **Checks:** a deadlock stop, input checks with clear error messages, and a report with IPC, branch, misprediction and violation counts.

3. Timing rules used

- ISSUE in cycle t, EX can start at t+1 if the operands are ready. A result broadcast (WB) in cycle w can be used by EX starting in w+1.
- Integer adder is not pipelined; FP adder and multiplier are pipelined. Load/store address calculation uses its own adder (1

cycle). The memory has one port and is not pipelined.
- WB takes up to CDB width results per cycle, oldest first. Commit is after WB. Stores and branches have no WB; a branch commits after its EX, a store after its address and data are ready, and then it uses the memory port.

4. Test cases and results

Every test was run at issue widths 1, 2, 3, 4 (`python3 tests/run_tests.py`). Each run was checked against a plain sequential interpreter (final registers, memory, and the order of committed instructions) and a timing checker (stage order, EX lengths, dependencies, one memory port, ROB size, issue / WB / commit limits). Full tables are in `tests/results/`.

Test	What it tests	w=1	w=2	w=3	w=4
t01 independent	no dependencies, 4 CDBs	18	13	12	11
t02 dependency chain	long chain + independent work	31	28	27	27
t03 structural	ROB of 4, 1 RS per unit	24	23	22	22
t04 branches	loop, taken / not taken Beq, mispredictions	41	33	33	31
t05 forwarding	store-to-load forwarding	22	17	17	17
t06 disambiguation	load passes a store, one violation	21	21	21	21
t07 handout sample	example input of the handout	10	8	8	8
t08 renaming	repeated writes of F2 and R1	24	24	22	22
t09 provided test	loop from the starter code	41	35	33	32
t10 CDB limit	8 independent FP adds, CDB = 1	12	12	12	12

(numbers are total cycles until the last commit)

Notes:

- **t01:** cycles fall as issue width grows, and the gain gets smaller (18 -> 13 -> 12 -> 11) because the dependent instructions at the end set a minimum length.
- **t02 / t03:** a dependency chain or a full ROB cannot be sped up by a wider issue, so the gain is small. This is the expected result.
- **t04:** checked by hand. The first `Bne` finishes EX in cycle 7, so the correct path is fetched in cycle 9 (n+2). Wrong-path instructions do not change registers or memory.
- **t05:** the load after `Sd F3, 0(R1)` gets its value in 1 MEM cycle (cycle 6), before the store has written memory (it writes at commit, cycle 9).
- **t06:** checked by hand. Two dependent adds with 3 EX cycles finish WB in 5 and 9, the store address is known in cycle 10, and the load that read the wrong value is found there and fetched again in cycle 12. Final values equal the sequential result. Width does not matter here because the program is limited by the slow add chain.
- **t10:** with one CDB the 8 results need 8 write-back cycles, so all widths take 12 cycles. With `CDB width = 4` the same program takes 12 / 8 / 7 / 6 cycles.
- **t07:** the handout example has `Bne R2 R3 -3` with R3 = 0, which loops forever, so I set R3 = 20 in my copy of the file.
- **Random testing:** `tests/fuzz.py` makes random programs with random hardware settings (loops, branches, forwarding, order

violations). 1300 programs x 4 widths were checked during development and 200 more at the end (`python3 tests/fuzz.py 200 9000`): no mismatch in registers, memory, or the timing rules. This included about 60 order violations and almost 900 mispredicted branches in the last run.

5. What works and what does not

Works:

- Issue width 1-4 for all instructions in the assignment (Add, Addi, Sub, Add.d, Sub.d, Mult.d, Ld, Sd, Beq, Bne), renaming, forwarding, speculation with recovery, in-order commit.
- Final registers and memory always equal those of the sequential execution.

Limits / not done:

- The 1-bit predictor is simple, so loops are mispredicted at the start and the end.
- Width-1 cycle counts can differ from the original starter code (see Introduction).
- Only the instructions listed above exist. Memory addresses must be between 0 and 255.
- The timing rules are my reading of the handout. If the grader’s reference has a slightly different rule (for example when a branch commits), cycle counts can differ by a cycle or two even though the results are correct.